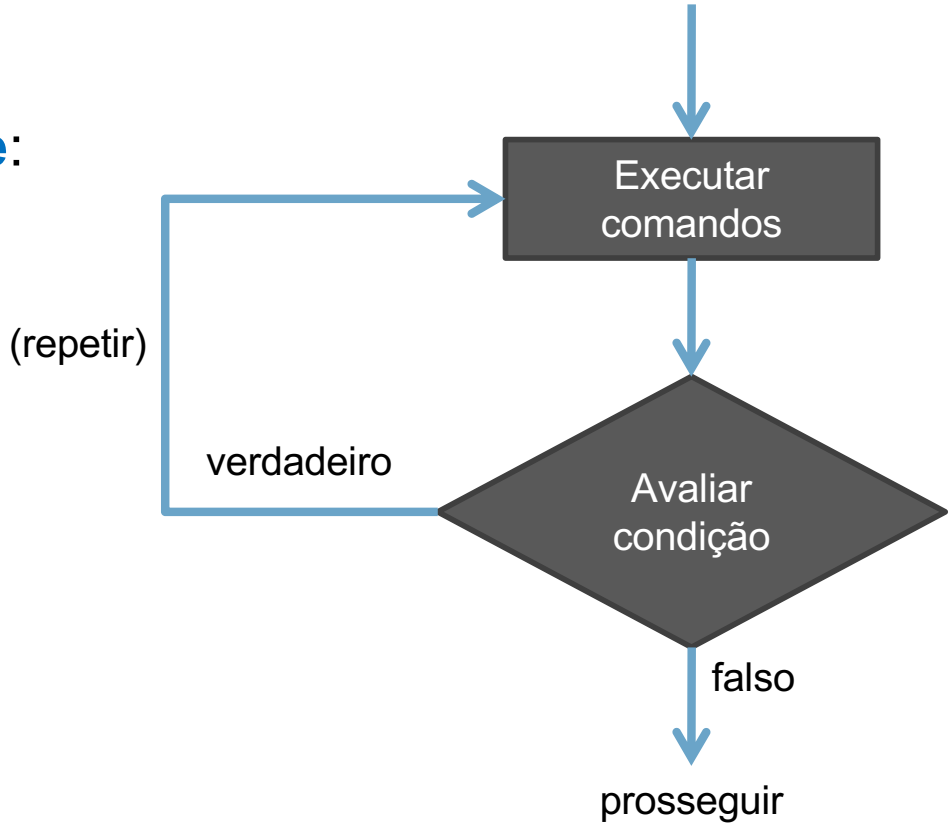
Comando `do-while`

Fluxograma do **do-while**:



- Os comandos serão executados **ao menos uma vez**, pois a avaliação da condição ocorre apenas a pós a primeira execução.

Comando do-while

- A estrutura **do-while** possui a seguinte sintaxe:

```
do {
    comando1;
    comando2;
    //...
} while(expessaoBoleana);
```

- A diferença do **while** para o **do-while** é que, no **do-while** sempre acontece a primeira execução do bloco de comandos e a expressão booleana só é avaliada ao final de cada execução.
  - Se for **verdadeira**, o bloco de comandos é repetido;
  - Se for **falso**, o laço termina.

Comando do-while

- Com o **do-while**, o programa anterior poderia ser escrito assim:

```c
#include <stdio.h>

int main()
{
    float nota1, nota2, nota3, media;
    char digito;

    do {
        //Lê as notas
        printf("Digite as tres notas do aluno: ");
        scanf("%f %f %f", &nota1, &nota2, &nota3);
        //Calcula a média
        media = (nota1 + nota2 + nota3)/3;
        //Imprime o resutado
        printf("A media eh: %f\n", media);
        //Deve continuar?
        printf("continuar? (s/n):");
        fflush(stdin);
        scanf("%c", &digito);
    } while (digito == 's');
}
```

Podemos utilizar expressões booleanas tão complexas quanto se queira dentro dos parênteses do **while**.

Obs.: Zero sempre é interpretado como falso e qualquer outro número como verdadeiro.

```
C:\CE\While\bin\Debug\While.exe
Digite as tres notas do aluno: 7.0 8.0 9.0
A media eh: 8.000000
continuar? (s/n):s
Digite as tres notas do aluno: 9.5 9.0 8.5
A media eh: 9.000000
continuar? (s/n):s
Digite as tres notas do aluno: 5.5 6.5 7.5
A media eh: 6.500000
continuar? (s/n):n
```

Comando do-while

- Variações do **do-while**:

```
do {
    comando1;
    comando2;
    //...
} while(expessaoBoleana);
```

```
do
    comando;
while (expressaoBoleana);
```

```
do comando; while (expressaoBoleana);
```

- As chaves são necessárias apenas quando há mais de um comando a ser repetido.

Comando do-while

- Exemplos de uso **do-while**:

```c
#define FALSE 0
#define TRUE 1

int terminar=FALSE;
int i=0;

do {
    printf("Continuando...\n")
    if (i>10) terminar = TRUE;
    i++;
} while (!terminar);
```

```c
int i=0;

do {
    printf("i: %d\n", i);
    i++;
} while (i<10);
```

```c
int counter = 5;
long factorial = 1;

do
{
    factorial *= counter--;
} while (counter > 1);
printf("%d", factorial);
```

Comando `do-while`

- Algumas vezes sabemos exatamente quantas vezes queremos executar os comandos.
- Nesses casos podemos utilizar uma variável inteira como contador, incrementando-a a cada execução, e verificando se a mesma atingiu valor desejado de execuções.
- Ex.: Imprimir todos os números de 1 a 15:

Inicialização

Avaliar condição

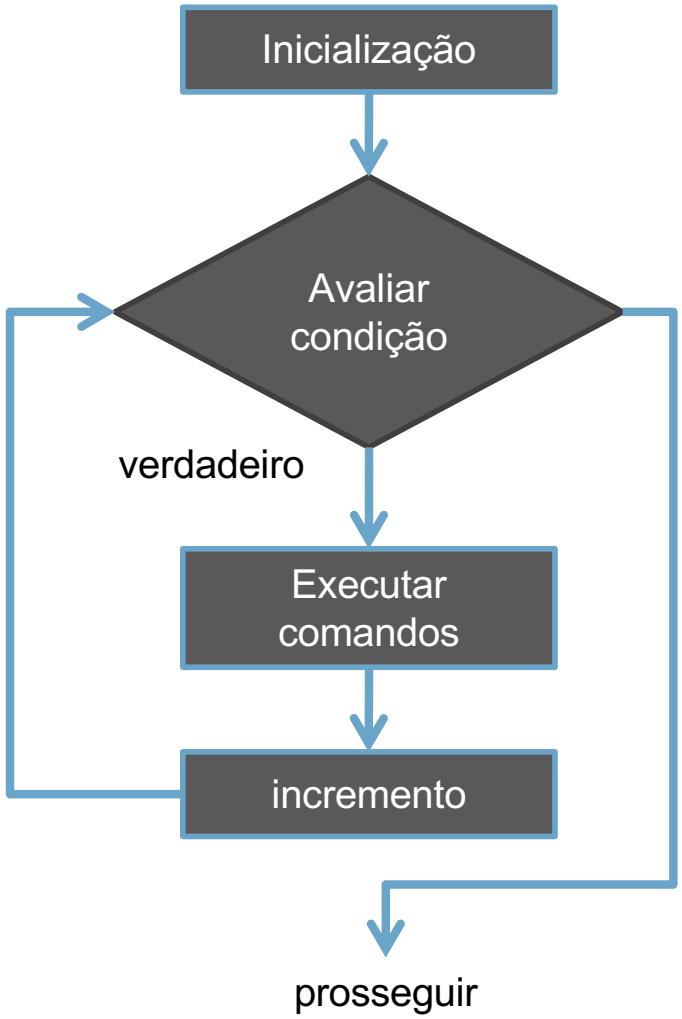
incremento

```c
#include <stdio.h>

int main()
{
    int i=1;

    while (i<=15) {
        printf("Execucao %d\n", i);
        i++;
    }

    printf("Fim.\n");
}
```

```
C:\CE\For\bin\Debug\For.exe
Execucao 1
Execucao 2
Execucao 3
Execucao 4
Execucao 5
Execucao 6
Execucao 7
Execucao 8
Execucao 9
Execucao 10
Execucao 11
Execucao 12
Execucao 13
Execucao 14
Execucao 15
Fim.
```

Comando for

- Fluxograma do **for**:



Comando for

- A estrutura **for** é especializada para estes casos.
- Sua sintaxe é a seguinte:

```
for (inicializacao; expressaoBoleana; incrementos) {
    comando1;
    comando2;
    //...
}
```

- No **for** há locais específicos para se definir:
- A **inicialização** (executada apenas antes do primeiro laço);
- A **expressão booleana** (avaliada sempre no começo de cada laço);
- Os **incrementos** (realizados apenas ao final de cada laço);
- Obs.: nenhum destes termos é obrigatório.
- Melhora a legibilidade do código, pois agrupa em um mesmo local a inicialização, o incremento a ser dado e a condição de término do laço.

Comando for

- A estrutura **for** é especializada para estes casos.
- Sua sintaxe é a seguinte:

```
for (inicializacao; expressaoBoleana; incrementos) {
    comando1;
    comando2;
    //...
}
```

- No **for** há locais específicos para se definir:
  - A **inicialização** (executada apenas antes do primeiro laço);
  - A **expressão booleana** (avaliada sempre no começo de cada laço);
  - Os **incrementos** (realizados apenas ao final de cada laço);
- Obs.: nenhum destes termos é obrigatório.
- Melhora a legibilidade do código, pois agrupa em um mesmo local a inicialização, o incremento a ser dado e a condição de término do laço.

Comando for

- Se soubermos de antemão quantos alunos há na turma (por exemplo: 5) , o programa anterior poderia ser escrito assim:

```c
#include <stdio.h>

int main()
{
    float nota1, nota2, nota3, media;
    int conta;

    for (conta=1;conta<=5;conta++) {
        //Lê as notas
        printf("Digite as tres notas do aluno %d: ", conta);
        scanf("%f %f %f", &nota1, &nota2, &nota3);
        //Calcula a média
        media = (nota1 + nota2 + nota3)/3;
        //Imprime o resutado
        printf("A media eh: %f\n", media);
    }
    printf("Fim.");
}
```

```
C:\CE\For\bin\Debug\For.exe
Digite as tres notas do aluno 1: 5.0 4.5 8.0
A media eh: 5.833333
Digite as tres notas do aluno 2: 8.5 9.0 7.5
A media eh: 8.333333
Digite as tres notas do aluno 3: 4.5 2.3 5.0
A media eh: 3.933333
Digite as tres notas do aluno 4: 4.0 5.0 6.0
A media eh: 5.000000
Digite as tres notas do aluno 5: 7.5 6.5 8.0
A media eh: 7.333333
Fim.
```

Comando for

- Variações do **for** (nem todas são recomendáveis):

```
for (inicializacao; expressaoBoleana; incrementos) comando;
```

```
inicializacao;
for (; expressaoBoleana; incrementos) {
    comandos;
}
```

```
inicializacao;
for (; expressaoBoleana; ) {
    comandos;
    incrementos;
}
```

```
for (;;) {
    //loop infinito
    comandos;
}
```

- As chaves são necessárias apenas quando há mais de um comando a ser repetido.

Comando for

- Outros exemplos de uso do **for**:

```
int i;

for (i=99; i>0; i-=2) {
    printf("i:%d\n", i);
}
```

```
int i=0;

for (; i<100; i=i+2) {
    printf("i:%d\n", i);
}
```

```
int i, j;

for (i=0, j=100; i<40 || j>60; i++, j--) {
    printf("i:%d j:%d\n", i, j);
}
```

Comando for

- Outros exemplos de uso do **for**:

```c
#include <stdio.h>
int main()
{
    int i;
    //Imprime toda a tabela ASCII:
    for (i=0; i<256; i++) {
        printf("O codigo ASCII de '%c' eh %d\n", i, i);
    }
}
```

```
C:\CE\For\bin\Debug\For.exe
O codigo ASCII de ' ' eh 0
O codigo ASCII de '☺' eh 1
O codigo ASCII de '☻' eh 2
O codigo ASCII de '♥' eh 3
O codigo ASCII de '♦' eh 4
O codigo ASCII de '♣' eh 5
O codigo ASCII de '♠' eh 6
O codigo ASCII de '' eh 7
O codigo ASCII de ' eh 8
O codigo ASCII de '     ' eh 9
O codigo ASCII de '
' eh 10
O codigo ASCII de '♂' eh 11
O codigo ASCII de '♀' eh 12
' eh 13o ASCII de '
O codigo ASCII de '♫' eh 14
O codigo ASCII de '☼' eh 15
O codigo ASCII de '►' eh 16
O codigo ASCII de '◄' eh 17
O codigo ASCII de '↕' eh 18
O codigo ASCII de '‼' eh 19
O codigo ASCII de '¶' eh 20
O codigo ASCII de '§' eh 21
O codigo ASCII de '▬' eh 22
O codigo ASCII de '↨' eh 23
```

```
C:\CE\For\bin\Debug\For.exe
O codigo ASCII de ' ' eh 32
O codigo ASCII de '!' eh 33
O codigo ASCII de '"' eh 34
O codigo ASCII de '#' eh 35
O codigo ASCII de '$' eh 36
O codigo ASCII de '%' eh 37
O codigo ASCII de '&' eh 38
O codigo ASCII de '' eh 39
O codigo ASCII de '(' eh 40
O codigo ASCII de ')' eh 41
O codigo ASCII de '*' eh 42
O codigo ASCII de '+' eh 43
O codigo ASCII de ',' eh 44
O codigo ASCII de '-' eh 45
O codigo ASCII de '.' eh 46
O codigo ASCII de '/' eh 47
O codigo ASCII de '0' eh 48
O codigo ASCII de '1' eh 49
O codigo ASCII de '2' eh 50
O codigo ASCII de '3' eh 51
O codigo ASCII de '4' eh 52
O codigo ASCII de '5' eh 53
O codigo ASCII de '6' eh 54
O codigo ASCII de '7' eh 55
O codigo ASCII de '8' eh 56
```

```
C:\CE\For\bin\Debug\For.exe
O codigo ASCII de '8' eh 56
O codigo ASCII de '9' eh 57
O codigo ASCII de ':' eh 58
O codigo ASCII de ';' eh 59
O codigo ASCII de '<' eh 60
O codigo ASCII de '=' eh 61
O codigo ASCII de '>' eh 62
O codigo ASCII de '?' eh 63
O codigo ASCII de '@' eh 64
O codigo ASCII de 'A' eh 65
O codigo ASCII de 'B' eh 66
O codigo ASCII de 'C' eh 67
O codigo ASCII de 'D' eh 68
O codigo ASCII de 'E' eh 69
O codigo ASCII de 'F' eh 70
O codigo ASCII de 'G' eh 71
O codigo ASCII de 'H' eh 72
O codigo ASCII de 'I' eh 73
O codigo ASCII de 'J' eh 74
O codigo ASCII de 'K' eh 75
O codigo ASCII de 'L' eh 76
O codigo ASCII de 'M' eh 77
O codigo ASCII de 'N' eh 78
O codigo ASCII de 'O' eh 79
O codigo ASCII de 'P' eh 80
```

```
C:\CE\For\bin\Debug\For.exe
O codigo ASCII de 'Þ' eh 232
O codigo ASCII de 'Ú' eh 233
O codigo ASCII de 'Û' eh 234
O codigo ASCII de 'Ù' eh 235
O codigo ASCII de 'ý' eh 236
O codigo ASCII de 'Ý' eh 237
O codigo ASCII de '¯' eh 238
O codigo ASCII de '´' eh 239
O codigo ASCII de '-' eh 240
O codigo ASCII de '±' eh 241
O codigo ASCII de '=' eh 242
O codigo ASCII de '¾' eh 243
O codigo ASCII de '¶' eh 244
O codigo ASCII de '§' eh 245
O codigo ASCII de '÷' eh 246
O codigo ASCII de '¸' eh 247
O codigo ASCII de '°' eh 248
O codigo ASCII de '¨' eh 249
O codigo ASCII de '·' eh 250
O codigo ASCII de '¹' eh 251
O codigo ASCII de '³' eh 252
O codigo ASCII de '²' eh 253
O codigo ASCII de '■' eh 254
O codigo ASCII de ' ' eh 255
```

Estruturas de Repetição: break

- O **break** permite encerrar um laço no meio de sua execução. Pode ser utilizado com
  - **while**, **do-while**, ou **for**
- A sintaxe é a seguinte:

```
while (expressaoBooleana1) {
   comandos1;
   if (expressaoBoolana2) break;
   comandos2;
}
```

```
do {
   comandos1;
   if (expressaoBoolana2) break;
   comandos2;
} while (expressaoBooleana1);
```

```
for (inicializacao; expressaoBooleana1; incrementos) {
     comandos1; //...
     if (expressaoBoolana2) break;
     comandos2; //...
}
```

  - Dificulta a compreensão do código, pois há mais de uma condição de término a ser verificada, mas é útil quando precisamos encerrar o laço **no meio do bloco**, evitando que alguns comandos sejam executados.

Estruturas de Repetição: break

- Exemplo

```
C:\CE\Break\bin\Debug\Break.exe
Digite o maximo de entradas: 10
Entre o valor 1 (-1 para finalizar): 2.0
Entre o valor 2 (-1 para finalizar): 4.0
Entre o valor 3 (-1 para finalizar): 6.0
Entre o valor 4 (-1 para finalizar): -1
Media = 4.000000
```

```c
1   #include <stdio.h>
2
3   int main()
4   {
5       float num,media,soma=0;
6       int i,n;
7       printf("Digite o maximo de entradas: ");
8       scanf("%d",&n);
9
10      for(i=1; i<=n; i++) {
11          printf("Entre o valor %d (-1 para finalizar): ",i);
12          scanf("%f",&num);
13          if(num<0.0) break; //termina o laço
14          soma=soma+num;
15      }
16      media=soma/(i-1);
17      printf("Media = %f\n", media);
18  }
```

Estruturas de Repetição: continue

- O **continue** permite passar para a próxima repetição no meio do laço.
  - Pode ser utilizado no **while**, **do-while**, ou no **for**.
- A sintaxe é a seguinte:

```
while (expressaoBooleana1) {
    comandos1;
    if (expressaoBoolana2) continue;
    comandos2;
}
```

```
do {
    comandos1;
    if (expressaoBoolana2) continue;
    comandos2;
} while (expressaoBooleana1);
```

```
for (inicializacao; expressaoBooleana
    comandos1; //...
    if (expressaoBoolana2) continue;
    comandos2; //...
}
```

  - Dificulta a compreensão do código, pois há mais de uma condição a ser verificada, mas é útil quando precisamos pular os próximos comandos **no meio do bloco**, indo direto para a próxima repetição.

Estruturas de Repetição: continue

- **Exemplo**: calcular o produto apenas dos números digitados que forem diferentes de zero:

```c
#include <stdio.h>

int main()
{
    int i,num,produto;
    for(i=1,produto=1; i<=5; i++) {
        printf("Entre o valor %d:",i);
        scanf("%d",&num);
        //se num==0 pula os proximos comandos
        if(num==0) continue;
        produto*=num;
        printf("Produto parcial: %d\n", produto);
    }
    printf("Produto final: %d\n",produto);
}
```

```
C:\CE\Continue\bin\Debug\Continue.ex
Entre o valor 1:2
Produto parcial: 2
Entre o valor 2:0
Entre o valor 3:3
Produto parcial: 6
Entre o valor 4:5
Produto parcial: 30
Entre o valor 5:2
Produto parcial: 60
Produto final: 60
```

Estruturas de Repetição: Laços Aninhados

- Todos os laços vistos podem ser aninhados, uns dentro dos outros.
  Exemplos:

```
while (expressaoBooleana1) {
    comandos1;
    while (expressaoBooleana2) {
        comandos2;
        comandos3;
    }
    comandos4;
}
```

```
while (expressaoBooleana1) {
    comandos1;
    do {
        comandos2;
        comandos3;
    } while (expressaoBooleana1);
    comandos4;
}
```

```
for (inicializacao; expressaoBooleana1; incrementos) {
    comandos1;
    while (expressaoBooleana2) {
        comandos2;
        comandos3;
    }
    comandos4;
}
```

Estruturas de Repetição: Laços Aninhados

- Exemplo:

```c
#include <stdio.h>

int main()
{
    int i, j;
    int total=0;

    for (i=0; i<2; i++) {
        for (j=0; j<3; j++) {
            total = total + 1;
            printf("i: %d  j: %d  total: %d\n", i, j, total);
        }
    }
    printf("O valor final de total e: %d\n", total);
}
```

```
C:\CE\LacosAninhados\bin\Debug\La
i: 0  j: 0  total: 1
i: 0  j: 1  total: 2
i: 0  j: 2  total: 3
i: 1  j: 0  total: 4
i: 1  j: 1  total: 5
i: 1  j: 2  total: 6
O valor final de total e: 6
```

- Qual o valor final de **total**, impresso pelo programa?

Estruturas de Repetição: Laços Aninhados

- Exemplo: **impedir entradas inválidas**, para calcular o produto apenas dos números digitados que forem diferentes de zero:

```c
#include <stdio.h>

int main()
{
    int i,num,produto;
    for(i=1,produto=1; i<=5; i++) {
        //Le a entrada e prossegue se não for zero:
        do {
            printf("Entre o valor %d:",i);
            scanf("%d",&num);
            if (num==0) printf("O valor nao pode ser zero!\n");
        } while (num==0);
        //Calcula o produto:
        produto*=num;
        printf("Produto parcial: %d\n", produto);
    }
    printf("Produto final: %d\n",produto);
}
```

```
C:\CE\Continue\bin\Debug\Continu
Entre o valor 1:2
Produto parcial: 2
Entre o valor 2:0
O valor nao pode ser zero!
Entre o valor 2:0
O valor nao pode ser zero!
Entre o valor 2:1
Produto parcial: 2
Entre o valor 3:3
Produto parcial: 6
Entre o valor 4:4
Produto parcial: 24
Entre o valor 5:1
Produto parcial: 24
Produto final: 24
```

Estruturas de Repetição: resumo

- Como escolher entre as estruturas de repetição?

```
if (numero de vezes é conhecido)  {
        usa-se o for
} else {
        if (teste precisa ser feito no início) {
                usa-se while
        } else {
                usa-se do-while
        }
}
```

- Porém, na maioria dos casos, pode-se escolher mais de uma opção para resolver cada tipo de problema e acaba sendo uma questão de preferência pessoal.

Exemplo de Cálculo de Maior ou Máximo

```c
#include <stdio.h>

int main(){

    //entrada
    int num_medalhas = 0;
    int paises = 100;

    //saida
    int max_medalhas = 0;

    int i = 1;
    for(;i<=paises;i++){

        printf("Digite o numero de medalhas do pais %d:",i);
        scanf("%d",&num_medalhas);
        if(num_medalhas > max_medalhas){
            max_medalhas = num_medalhas;
        }
    }

    printf("O maior numero de medalhas foi:%d",max_medalhas);

    return 0;
}
```

Exemplo de Cálculo da Média

```c
#include <stdio.h>

int main(){

    //entrada
    int nota = 0;
    int num_notas = 5;

    //saida
    float media = 0;

    int soma_notas = 0;

    int i = 1;
    for(;i<=num_notas;i++){
        printf("Digite a nota do arbitro %d:",i);
        scanf("%d",&nota);
        soma_notas = soma_notas + nota;
    }

    media = (float)soma_notas/num_notas;
    printf("O media das notas foi %.2f",media);

    return 0;
}
```

Exemplo de Cálculo da Média

```c
#include <stdio.h>

int main(){

    //entrada
    int nota = 0;
    int num_notas = 5;

    //saida
    float media = 0;

    int soma_notas = 0;

    int i = 1;
    for(;i<=num_notas;i++){
        printf("Digite a nota do arbitro %d:",i);
        scanf("%d",&nota);
        soma_notas = soma_notas + nota;
    }

    media = (float)soma_notas/num_notas;
    printf("O media das notas foi %.2f",media);

    return 0;
}
```

Atividade 1: 1EE – 2014.1

1º quesito: Um revista especializa deseja calcular estatísticas sobre a primeira fase da copa do mundo do Brasil. Com o objetivo de automatizar este cálculo, você deverá criar um programa na linguagem C para ajudar os repórteres da publicação. A primeira fase da competição possui 34 times. O programa deverá ler os dados dos 34 times: 1) Número de vitórias; 2) Número de derrotas; 3) Número de empates; 4) Número de gols efetuados; 5) Número de gols sofridos; 6) Quantidade de cartões vermelhos recebidos (Expulsões).

O programa deverá exibir na tela:

a) O **maior** número de vitórias;
b) O **maior** número de derrotas;
c) O saldo **médio** de gols geral (o saldo de gols de um time = quantidade de gols efetuados – quantidade de gols recebidos);
d) Percentagem de times **com mais de** 3 cartões vermelhos.

Obs.: É proibido o uso de comando goto;
Os valores devem ser impressos na tela com precisão de **duas casas decimais** depois da virgula.

Atividade 3

- Faça um programa para imprimir a tabuada de 1 a 9 utilizando laços de repetição. A saída do programa deve ser do tipo:

```
1x1 = 1
1x2 = 2
...
1x9 = 9
          //linha em branco
2x1 = 2
2x2 = 4
...
2x9 = 18
          //linha em branco
3x1 = 3
3x2 = 6
...
...
9x1 = 9
...
9x9 = 81
```

Atividade 4

1. Escreva um algoritmo que lê 50 números inteiros e em seguida mostra a soma de todos os ímpares lidos.
2. Altere o algoritmo anterior para que ele considere apenas a soma dos ímpares que estejam entre 100 e 200.
3. Construa um algoritmo que leia um conjunto de 20 números inteiros e mostre qual foi o maior e o menor valor fornecido.
4. Altere o programa anterior para que ele não permita a entrada de valores negativos.

Atividade 5

- Fazer um programa para calcular a raiz quadrada de um número positivo usando o roteiro abaixo (baseado no método de aproximações sucessivas de Newton-Raphson).
- Seja *Y* o número para o qual se quer computar a raiz quadrada:
  - A primeira aproximação para a raiz quadrada de Y é dada por: $X_1 = \frac{Y}{2}$
  - As demais aproximações são calculadas pela seguinte fórmula de recorrência:

$$X_{i+1} = X_i - \frac{X_i^2 - Y}{2X_i}, \text{onde } X_i \text{ é a aproximação anterior e } X_{i+1} \text{ a nova.}$$

  - A aproximação deve continuar até que o valor obtido mude pouco com o passar das iterações (menos de *e* = 0.1), Ou seja:

$$\left| X_{i+1} - X_i \right| < e, \text{onde } e = 0.1$$

  - Compare o resultado obtido com o resultado produzido pela função **sqrt()** disponível na biblioteca <math.h>:

$$\left| X_n - sqrt(Y) \right|, \text{onde } n \text{ é o número de iterações realizadas}$$

  - Obs.: O valor absoluto | . | pode ser calculado através da função **fabs()**, disponível na biblioteca <math.h>.